

Дисциплина: Программирование корпоративных систем

Практическое занятие №4

Тестирование проекта

Отчет по тестированию проекта

Тайм-трекер

Консольное приложение для учета рабочего времени

Выполнил: Лобин Александр Ильич

Группа: ЭФБО-06-24

Москва, 2026

1. Цель работы

Цель практического занятия - разработать систему unit-тестов и тестовых сценариев для проекта "Тайм-трекер", подключить библиотеку тестирования к CMake-сборке и проверить основные функции классов приложения.

2. Краткое описание проекта

Тайм-трекер - консольное C++ приложение для учета рабочего времени по задачам. Пользователь может создавать задачи, запускать и останавливать рабочую сессию, сохранять данные локально и получать отчет о затраченном времени.

2.1 Тестируемые классы

Класс	Основные функции	Файл тестов
Task	Хранение данных задачи, преобразование статуса.	tests/test_task.cpp
TimeSession	Хранение данных сессии, расчет длительности.	tests/test_time_session.cpp
TaskManager	Создание, поиск, архивирование и загрузка задач.	tests/test_task_manager.cpp
TimeTracker	Запуск и остановка таймера, контроль активной сессии.	tests/test_time_tracker.cpp
Storage	Сохранение и загрузка задач и сессий.	tests/test_storage.cpp
ReportService	Расчет суммарного времени и вывод отчета.	tests/test_report_service.cpp
ConsoleUI	Работа меню и пользовательских команд.	tests/test_console_ui.cpp

3. Подключение Catch2

Для unit-тестирования используется библиотека Catch2. Она подключена в сборку через CMake FetchContent в файле tests/CMakeLists.txt.

```
include(FetchContent)
FetchContent_Declare(
    Catch2
    GIT_REPOSITORY https://github.com/catchorg/Catch2.git
    GIT_TAG v3.5.3
)
FetchContent_MakeAvailable(Catch2)
```

Основная логика проекта вынесена в библиотеку time_tracker_lib. Исполняемый файл приложения и тесты подключают эту библиотеку отдельно.

```
add_library(time_tracker_lib ...)
add_executable(time_tracker src/main.cpp)
target_link_libraries(time_tracker PRIVATE time_tracker_lib)
add_executable(unit_tests ...)
target_link_libraries(unit_tests PRIVATE time_tracker_lib Catch2::Catch2WithMain)
```

4. Unit-тесты

Для каждого основного класса разработан набор тестов, покрывающих нормальные сценарии, граничные случаи и некорректные входные данные.

4.1 Примеры проверок

- **Task** : преобразование статуса в строку, разбор строки, обработка неизвестного статуса, изменение статуса.
- **TimeSession** : точная длительность завершенной сессии, нулевая длительность, активная сессия, рост длительности во времени.
- **TaskManager** : добавление задач, последовательность идентификаторов, поиск существующей и отсутствующей задачи, архивирование.
- **TimeTracker** : запуск по активной задаче, отказ для отсутствующей или архивной задачи, запрет второй активной сессии, остановка таймера.
- **Storage** : пустое хранилище, сохранение задач, архивный статус, несколько задач, сохранение сессий, перезапись данных.
- **ReportService** : пустой отчет, сумма по одной задаче, сумма нескольких сессий, разделение задач, игнорирование активных сессий.
- **ConsoleUI** : выход и сохранение, добавление задачи, просмотр списка, архивирование, запуск и остановка таймера, неизвестная команда.

5. Тестовые сценарии

Кроме unit-тестов реализованы отдельные мини-программы, раскрывающие основные этапы работы приложения. Они находятся в каталоге `tests/scenarios/` и собираются отдельными исполняемыми файлами.

Сценарий	Файл	Проверяемый процесс
Базовый рабочий процесс	<code>scenario_basic_workflow.cpp</code>	Создание задачи, запуск таймера, остановка таймера.
Сохранение и загрузка	<code>scenario_persistence.cpp</code>	Запись данных в файлы и повторная загрузка.
Формирование отчета	<code>scenario_reporting.cpp</code>	Расчет суммарного времени по задачам и вывод отчета.

6. Команды сборки и запуска

Проект собирается из отдельной директории `build`. При первой конфигурации CMake загружает Catch2.

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

Сценарные программы после сборки находятся в каталоге `build/tests/scenarios`.

```
./tests/scenarios/scenario_basic_workflow
./tests/scenarios/scenario_persistence
./tests/scenarios/scenario_reporting
```

7. Ожидаемые результаты

- Все unit-тесты Catch2 должны завершаться успешно.
- Сценарий базового процесса должен вывести `Basic workflow passed`.
- Сценарий сохранения должен вывести `Persistence scenario passed`.
- Сценарий отчетов должен вывести отчет и строку `Reporting scenario passed`.

8. Вывод

В рамках практической работы к проекту подключена библиотека Catch2, настроена сборка unit-тестов через CMake, добавлены тесты для основных классов и реализованы отдельные мини-программы тестовых сценариев. Такая структура позволяет проверять как отдельные функции классов, так и основные пользовательские процессы приложения.

